

Blog Platform with Comments - Source Code

■■ PDF ■■■■ ■■■■■■ ■■ ■■■ ■■■■■■ ■■ ■■■■ ■■■■ ■■■■ Node.js / Express.js ■■ MongoDB
(Mongoose) ■■ ■■■■■■ ■■■■■■ ■■■■■■■■■■ ■■■■ ■■ ■■■ ■■■■■ ■■ ■■ ■■■

1. Mongoose Database Schemas (Models)

```
const mongoose = require('mongoose');

const UserSchema = new mongoose.Schema({
  username: { type: String, required: true, unique: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true }
});

const PostSchema = new mongoose.Schema({
  title: { type: String, required: true },
  content: { type: String, required: true },
  author: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  { timestamps: true }
});

const CommentSchema = new mongoose.Schema({
  post: { type: mongoose.Schema.Types.ObjectId, ref: 'Post', required: true },
  author: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  content: { type: String, required: true },
  { timestamps: true }
});

module.exports = {
  User: mongoose.model('User', UserSchema),
  Post: mongoose.model('Post', PostSchema),
  Comment: mongoose.model('Comment', CommentSchema)
};
```

2. JWT Authentication Middleware

```
const jwt = require('jsonwebtoken');

module.exports = function (req, res, next) {
  const token = req.header('Authorization');
  if (!token) return res.status(401).json({ message: 'Access Denied' });

  try {
    const verified = jwt.verify(token.split(" ")[1], 'JWT_SECRET_KEY');
    req.user = verified;
    next();
  } catch (err) {
    res.status(400).json({ message: 'Invalid Token' });
  }
};
```

3. Express App Server & RESTful APIs

```

const express = require('express');
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const { User, Post, Comment } = require('./models/Schemas');
const verifyToken = require('./middleware/auth');

const app = express();
app.use(express.json());

// --- AUTHENTICATION ---
app.post('/api/auth/register', async (req, res) => {
  try {
    const hashedPassword = await bcrypt.hash(req.body.password, 10);
    const newUser = new User({ username: req.body.username, email: req.body.email, password: hashedPassword });
    await newUser.save();
    res.status(201).json("User registered!");
  } catch (err) { res.status(500).json(err.message); }
});

app.post('/api/auth/login', async (req, res) => {
  const user = await User.findOne({ email: req.body.email });
  if (!user) return res.status(400).json("User not found");
  const validPass = await bcrypt.compare(req.body.password, user.password);
  if (!validPass) return res.status(400).json("Invalid password");
  const token = jwt.sign({ _id: user._id }, 'JWT_SECRET_KEY');
  res.json({ token, username: user.username });
});

// --- POSTS CRUD ---
app.get('/api/posts', async (req, res) => {
  const posts = await Post.find().populate('author', 'username');
  res.json(posts);
});

app.post('/api/posts', verifyToken, async (req, res) => {
  const newPost = new Post({ title: req.body.title, content: req.body.content, author: req.user._id });
  await newPost.save();
  res.status(201).json(newPost);
});

app.put('/api/posts/:id', verifyToken, async (req, res) => {
  const post = await Post.findById(req.params.id);
  if (post.author.toString() !== req.user._id) return res.status(403).json("Unauthorized");
  post.title = req.body.title || post.title;
  post.content = req.body.content || post.content;
  await post.save();
  res.json(post);
});

app.delete('/api/posts/:id', verifyToken, async (req, res) => {
  const post = await Post.findById(req.params.id);
  if (post.author.toString() !== req.user._id) return res.status(403).json("Unauthorized");
  await post.deleteOne();
  res.json("Post deleted");
});

// --- COMMENTS SYSTEM ---
app.get('/api/posts/:postId/comments', async (req, res) => {
  const comments = await Comment.find({ post: req.params.postId }).populate('author', 'username');
  res.json(comments);
});

app.post('/api/posts/:postId/comments', verifyToken, async (req, res) => {
  const newComment = new Comment({ post: req.params.postId, author: req.user._id, content: req.body.content });
  await newComment.save();
  res.status(201).json(newComment);
});

mongoose.connect('mongodb://localhost:27017/blogDB').then(() => {
  app.listen(5000, () => console.log('Server open on port 5000'));
});

```